

多人脸采集素材自动化清洗系统需求分析报告

1 项目概述

1.1 项目背景与建设目标

在车载座舱多人脸素材采集业务中（如车载摄像头路测数据采集、网约车安全监控数据归档、驾驶员行为分析数据集构建），采集团队每日产生大量包含多名乘员的座舱图像。这些原始素材在进入算法训练、质量审核或合规归档环节之前，必须完成人脸定位、身份归类、座位标注等数据清洗工作。当前行业通行做法为人工逐张标注，单张含 4-5 名乘员的图像平均标注耗时约 45 秒，且人工标注的身份一致性率仅为 70%-80%，错标、漏标率居高不下，严重制约了数据处理效率和下游模型训练质量。

本项目的建设目标为：开发一套基于本地 Web 服务的多人脸采集素材自动化清洗系统，实现从图像上传到人脸检测、跨图片身份匹配、座位区域分类、可视化标注及统计报告生成的全链路自动化，将人工介入降至最低。系统须满足以下核心指标：多人脸检测准确率不低于 95%，座位分类准确率不低于 90%，跨图片身份匹配一致率不低于 80%，单张图片处理耗时不超过 5 秒（CPU 推理环境），前端界面支持移动端访问且无需安装任何客户端软件。

1.2 项目适用场景

本系统适用于以下典型业务场景：

车载路测数据清洗：智能座舱研发过程中，测试车辆在道路实测阶段由车内摄像头持续采集包含驾驶员、副驾驶及后排乘客的图像素材。数据工程师需要将这些素材快速清洗、分类、标注后交付给算法团队用于驾驶员状态识别、乘员行为分析等模型训练。

网约车安全监控数据归档：网约车平台的车内安全监控摄像头每日产生大量乘员图像。质检团队需要对抽检数据进行乘员人数确认、座位分布记录和驾驶员身份核验，自动化标注可大幅减少人工审核工作量。

驾考考试数据采集：科目三路考中，车内摄像头采集包含考生、安全员及后排乘员的图像。系统可自动标注考生身份和座位位置，辅助考试成绩审核和数据归档。

算法研发数据集构建：AI 算法团队在构建车载场景人脸识别训练集时，需要对大规模采集数据进行自动化预标注（人脸框、身份标签、座位标签），再交由人工进行精修审核，从而显著降低标注成本。

1.3 项目核心价值

本系统的核心价值体现在以下四个方面。

第一，**全链路自动化**。系统将人脸检测、特征提取、身份匹配、座位分类、可视化标注、统计报告生成六个环节整合为端到端自动化流水线，用户仅需上传图片即可获得完整清洗结果，无需手工标注。

第二，**双引擎降级保障**。人脸检测和特征匹配两个核心环节均设计了主方案与降级方案的双引擎架构。当高级模型（YOLOv8-face、InsightFace ArcFace）不可用时，系统自动降级至内置的轻量级方案（OpenCV Haar Cascade、像素统计特征），确保在任何环境下均可运行，无需额外下载模型文件。

第三，**本地化数据隐私保护**。所有图像处理均在本地完成，不依赖云端服务，不向任何外部系统传输用户数据，从根本上满足车载场景中对乘客隐私保护的合规要求。

第四，**零安装门槛**。系统以本地 Web 服务形式运行，用户通过浏览器即可访问，支持拖拽上传、移动端适配，无需安装任何桌面客户端或标注工具。

1.4 项目仓库与技术基线说明

项目托管于 GitHub 开源仓库，仓库地址为 `https://github.com/Nozomi9967/face-detect`，核心应用代码位于仓库的 `face-detect-app/` 子目录下。

系统采用 Python 语言开发后端服务，使用原生 HTML5 + CSS3 + JavaScript 实现前端界面，不依赖任何前端构建工具或框架。核心技术基线如下表所示：

技术层次	主方案	降级方案	说明
人脸检测	YOLOv8-face (ultralytics 库)	OpenCV Haar Cascade	YOLOv8-face 精度更高； Haar Cascade 无需下载模型文件
人脸特征匹配	InsightFace ArcFace 512 维嵌入向量	像素统计特征（16x16 灰度图展平 + L2 归一化，256 维）	ArcFace 匹配精度更高；像素特征完全离线可用
座位区域分类	基于归一化坐标的规则引擎	无降级方案（规则引擎本身即为完整方案）	5 个座位区域，按坐标区间判定
可视化标注	PIL (Pillow) 图像库 + 系统中文字体	无降级方案	支持中文标注文字渲染
后端服务框架	FastAPI + Uvicorn	无降级方案	ASGI 高性能 Web 框架
前端界面	HTML5 + CSS3 + Vanilla JavaScript	无降级方案	单文件实现，响应式布局

2 总体业务需求

2.1 核心业务功能总览

本系统提供以下核心业务功能，按功能性质划分为六大模块：

模块一：多人脸检测。系统自动检测上传图片中的所有人脸，返回每个人脸的边界框坐标（像素坐标 `[x1, y1, x2, y2]`）和检测置信度分数，支持同时检测 5 张及以上人脸。

模块二：人脸特征提取与身份匹配。系统为每张检测到的人脸提取特征向量，通过余弦相似度计算实现跨图片的身份聚类匹配，为同一人在不同图片中分配一致的唯一 `person_id`。

模块三：座位区域自动分类。系统根据人脸边界框中心点在图像中的归一化坐标位置，自动将其分类到预定义五个座位区域之一（主驾、副驾、后排左、后排中、后排右）。

模块四：可视化标注输出。系统在原始图像上绘制彩色边界框、座位名称标签（中文）、人员 ID 和检测置信度，不同座位区域使用不同颜色区分，输出 Base64 编码的 JPEG 标注图像。

模块五：单图检测与批量清洗。系统提供单张图片即时检测和批量多图片统一清洗两种工作模式。批量模式下额外生成跨图片的人员统计报告。

模块六：Web 交互界面。系统提供基于浏览器的响应式单页应用，支持拖拽上传、结果预览、模式切换等操作，适配桌面端和移动端。

2.2 业务流程1：单张图片人脸检测可视化流程

单张图片检测的业务流程描述如下。

用户在浏览器中访问系统首页，默认进入“单张检测”模式。用户点击上传区域选择一张图片或拖拽一张图片到上传区域，前端通过 HTTP POST 请求（multipart/form-data 编码，字段名为 `file`）将图片文件发送至后端 `/api/detect` 接口。

后端接收到图片文件后，首先使用 OpenCV 的 `cv2.imdecode` 函数将图片字节流解码为 numpy 数组（BGR 色彩空间）。随后调用 Pipeline 流水线引擎执行处理。流水线依次调用 FaceDetector 进行人脸检测（自动选择 YOLOv8-face 或 Haar Cascade 引擎），对每个检测到的人脸调用 FaceRecognizer 提取特征向量，调用 FaceRecognizer 的身份匹配逻辑分配 `person_id`，调用 SeatClassifier 进行座位区域分类，最后调用 FaceAnnotator 在原始图像上绘制标注。

处理完成后，后端将标注图像编码为 JPEG 格式（质量参数 85），再转为 Base64 字符串，组装为 JSON 响应返回前端。响应体包含 `image_id`、`filename`、`num_faces`（检测到的人脸总数）、`faces` 数组（每个元素包含 `bbox`、`conf`、`seat`、`seat_label`、`person_id` 五个字段）、`annotated_image_base64`（Base64 编码的标注图像）和 `processing_time_ms`（处理耗时，单位为毫秒）。

同时，后端将上传的原始图片保存至 `data/uploads/` 目录（文件名格式为 `{时间戳毫秒数}{扩展名}`），将标注结果图片保存至 `data/results/` 目录（文件名格式为 `result_{时间戳毫秒数}{扩展名}`）。

前端接收到响应后，渲染结果展示卡片：顶部显示标注图像预览，旁边显示人脸数量徽章和处理耗时徽章，下方以列表形式展示每个人脸的座位标签（带颜色标记）和检测置信度（以进度条形式展示）。

2.3 业务流程2：批量图片素材自动清洗、跨图人脸匹配、统计报告生成流程

批量清洗模式的业务流程描述如下。

用户在界面上切换到“批量清洗”模式，此时上传区域允许同时选择多张图片。用户选择多张图片后，前端通过 HTTP POST 请求（multipart/form-data 编码，字段名为 `files`）将所有图片文件一次性发送至后端 `/api/detect-batch` 接口。

后端的批量处理流程分为四个阶段。

阶段一：逐图检测与特征提取。后端遍历所有上传图片，对每张图独立执行图像解码、人脸检测和特征向量提取。所有检测到的人脸信息被收集到一个全局列表中，每个人脸记录标记了所属的 `image_id`（从 0 开始递增的图片序号）。

阶段二：全局跨图身份匹配。后端将全局人脸列表传入 FaceRecognizer 的 `match_faces_across_images` 方法。该方法维护一个 `person_db` 字典（键为 `person_id`，值为已知人员的特征向量列表），按图片顺序逐一遍历所有人脸，计算当前人脸特征与 `person_db` 中每个已知身份的余弦相似度。若最高相似度达到或超过阈值（默认 0.5），则将对应的 `person_id` 分配给当前人脸；否则创建新的 `person_id`（从 1 开始递增）。此阶段确保了同一人在不同图片中获得一致的 ID。

阶段三：逐图座位分类与标注。后端按 `image_id` 将全局人脸列表拆分回各图片维度，对每张图片独立执行座位区域分类和可视化标注，生成各自的标注图像。

阶段四：聚合统计与响应组装。后端按 `person_id` 对全部人脸进行分组统计，计算每个 `person_id` 的出现次数（`appearance_count`，即在多少张图片中出现）和主要座位（`primary_seat`，即出现次数最多的座位区域）。统计结果组装为 `aggregate_stats` 数组。最终响应包含 `total_images`（图片总数）、`total_faces`（人脸总数）、`processing_time_ms`（总耗时）、`results`（逐图结果数组，每个元素结构同单图检测响应）和 `aggregate_stats`（聚合统计数组）。

批量结果图片保存至 `data/results/` 目录，文件名格式为 `batch_{时间戳毫秒数}_{图片序号}.jpg`。

前端接收到批量响应后，渲染聚合统计表格（展示人员 ID、出现次数、主要座位）和逐图结果卡片（每张图的标注图和人脸列表）。

2.4 业务流程3：人脸座位区域自动分类标注流程

座位区域自动分类的业务流程嵌入在上述两条主流程之中，作为流水线的关键环节执行。

当 FaceDetector 完成人脸检测并输出边界框坐标后，SeatClassifier 对每个人脸执行如下判定逻辑：首先计算人脸边界框的中心点坐标，公式为 $cx = (x1 + x2) / 2$ ， $cy = (y1 + y2) / 2$ ；然后将中心点坐标归一化到 $[0, 1]$ 范围，公式为 $nx = cx / \text{图像宽度}$ ， $ny = cy / \text{图像高度}$ ；最后按照预定义五个矩形区域规则，依次判断归一化坐标 (nx, ny) 落入哪个区域，返回对应的座位 ID 和中文标签。

图像坐标系的约定为：Y 轴上方（ny 值较小）代表车辆后排，Y 轴下方（ny 值较大）代表车辆前排；X 轴左侧（nx 值较小）代表车辆左侧，X 轴右侧（nx 值较大）代表车辆右侧。

分类完成后，FaceAnnotator 根据座位 ID 从颜色映射表中获取对应的 RGB 颜色值，在原始图像上绘制与座位颜色一致的矩形边界框（线宽 3 像素），在框上方绘制填充色背景条并标注白色中文文字（格式为“{座位标签} ID:{person_id}”），在框下方标注检测置信度数值。

3 功能性需求

3.1 人脸检测模块需求

3.1.1 主模型 YOLOv8-face 检测需求

FR-DET-001（强需求）：系统必须优先使用 YOLOv8-face 作为人脸检测引擎。YOLOv8-face 模型通过 ultralytics 库加载，模型文件名默认为 `yolov8n-face.pt`。

业务作用：YOLOv8-face 是当前开源社区中精度领先的人脸检测模型之一，在 WIDER FACE 等基准数据集上的检测精度优于传统 Haar Cascade 和 MTCNN 方案，能够适应车载场景中光照变化、部分遮挡和多种人脸角度。

FR-DET-002（强需求）：YOLOv8-face 模型的加载路径查找必须按以下顺序执行：首先检查当前工作目录下的 `yolov8n-face.pt` 文件是否存在；若不存在，检查用户目录下的 `.ultralytics/weights/yolov8n-face.pt` 路径；若均不存在，触发降级流程。

业务作用：提供灵活的模型文件查找策略，用户可将模型文件放置在不同位置而不影响系统正常运行。

FR-DET-003（强需求）：YOLOv8-face 引擎的检测置信度阈值必须可配置，默认值为 0.5。低于阈值的人脸检测结果应被过滤丢弃。推理必须在 CPU 设备上执行（`device="cpu"`），推理过程中不输出冗余日志（`verbose=False`）。

业务作用：可调节的置信度阈值允许用户根据场景需求在召回率和精确率之间取得平衡。强制 CPU 推理确保系统无需 GPU 即可运行。

FR-DET-004（强需求）：YOLOv8-face 引擎的检测结果必须包含每个人脸的四元素边界框坐标 [`x1`, `y1`, `x2`, `y2`]（左上角和右下角的像素坐标，数据类型为浮点数）和检测置信度分数（0 到 1 之间的浮点数）。

业务作用：标准化的输出格式为下游身份匹配、座位分类和标注绘制提供统一的数据接口。

3.1.2 降级方案 OpenCV Haar Cascade 兜底需求

FR-DET-005（强需求）：当 ultralytics 库未安装（触发 ImportError）或 YOLOv8-face 模型文件（`yolov8n-face.pt`）在预设路径中均不存在时，系统必须自动降级至 OpenCV 内置的 Haar Cascade 分

类器，使用 `haarcascade_frontalface_default.xml` 模型文件。降级过程不得导致系统启动失败或抛出未捕获异常。

业务作用：保证系统在无外部模型依赖的干净环境下仍可运行基本的人脸检测功能，降低部署复杂度。

FR-DET-006 (强需求)：Haar Cascade 引擎的检测参数必须为：灰度图转换（`cv2.cvtColor` BGR 转 GRAY）、多尺度检测参数 `scaleFactor=1.1`、`minNeighbors=5`、`minSize=(60, 60)`、检测标志 `flags=cv2.CASCADE_SCALE_IMAGE`。

业务作用：固定检测参数确保 Haar Cascade 引擎在车载座舱图像上的检测效果稳定可预期。`minSize` 设为 60x60 像素可过滤过小的误检区域。

FR-DET-007 (强需求)：Haar Cascade 引擎必须为每个检测到的人脸计算伪置信度分数。计算公式为：
`conf = min(0.5 + area_ratio * 10, 0.99)`，其中 `area_ratio = (人脸框宽度 × 人脸框高度) / (图像宽度 × 图像高度)`。

业务作用：Haar Cascade 本身不输出置信度分数，通过人脸面积占比构造伪置信度，保持与 YOLOv8-face 引擎输出格式的一致性，使下游模块无需区分检测引擎类型。

FR-DET-008 (强需求)：Haar Cascade 模型文件加载失败时（`cv2.CascadeClassifier.load()` 返回空），系统必须抛出 `RuntimeError` 异常并附带错误消息 "无法加载 Haar Cascade 模型：
{cascade_path}"。

业务作用：在两种检测引擎均不可用的极端情况下，给出明确的错误提示，便于运维人员快速定位问题。

3.1.3 人脸框坐标输出、标注绘制基础需求

FR-DET-009 (强需求)：无论使用哪种检测引擎，最终输出的人脸列表必须按边界框的 x1 坐标（左上角横坐标）从小到大排序，即从左至右排列。

业务作用：统一的排序规则便于前端展示和后续处理，使用户在查看结果时获得一致的视觉体验。

FR-DET-010 (强需求)：系统必须通过 `is_yolo` 属性对外暴露当前使用的检测引擎类型（返回 `True` 表示 YOLOv8-face，返回 `False` 表示 Haar Cascade），供日志记录和调试使用。

业务作用：为开发人员和运维人员提供引擎状态查询接口，便于确认系统当前运行在哪个检测引擎上。

3.2 人脸特征匹配模块需求

3.2.1 ArcFace 512 维人脸特征提取与相似度匹配需求

FR-REC-001 (弱需求/可选扩展)：系统的人脸识别引擎模块（`recognizer.py`）设计上支持 InsightFace ArcFace 方案作为主方案，提取 512 维人脸嵌入向量用于身份匹配。InsightFace 依赖需在 `requirements.txt` 中声明（`insightface>=0.7.3`）。

业务作用：ArcFace 512 维嵌入向量在大规模人脸验证任务上具有极高的区分度（LFW 基准精度 99.83%），可确保跨图片身份匹配的高准确率。

FR-REC-002 (弱需求/可选扩展)：当 InsightFace 库可用且 ArcFace 模型已下载时，系统应优先使用 ArcFace 引擎提取 512 维特征向量。匹配时使用余弦相似度度量，相似度超过可配置阈值（默认 0.5）的两张人脸应判定为同一人。

业务作用：提供高精度的人脸特征匹配能力，适应人脸角度变化、光照差异和部分遮挡等复杂场景。

3.2.2 降级方案像素直方图统计特征匹配需求

FR-REC-003 (强需求)：系统必须实现基于纯 OpenCV 的像素统计特征提取方案作为降级方案，且当前版本实际使用该方案。特征提取流程为：将人脸边界框内的图像区域裁剪为 ROI → 缩放至 16x16 像素 → 转换为灰度图 → 执行直方图均衡化（`cv2.equalizeHist`）→ 展平为 float32 类型的一维数组（256 维）→ L2 归一化。

业务作用：像素统计特征方案完全依赖 OpenCV 基础功能，无需下载任何外部模型文件，确保系统在任何环境下均可完成基本的身份匹配。

FR-REC-004 (强需求)：余弦相似度计算必须为静态方法，输入两个特征向量，输出浮点型相似度分数。当任一输入为 None 时返回 0.0。计算公式为 `np.dot(emb1, emb2)`（因向量已 L2 归一化，点积即余弦相似度）。

业务作用：标准化的相似度计算接口，确保匹配结果的一致性。

FR-REC-005 (强需求)：特征提取时必须对边界框坐标进行图像边界钳位（clamp），确保裁剪区域不超出图像范围。若裁剪后的 ROI 为空（宽度或高度为 0），必须返回 None 而非抛出异常。

业务作用：防止边界框位于图像边缘时出现越界错误，保证流水线的鲁棒性。

3.2.3 批量图片同一人脸跨图聚类匹配需求

FR-REC-006 (强需求)：系统必须实现跨图片的人脸身份匹配方法 `match_faces_across_images`。该方法接收包含所有图片全部人脸信息的列表（每个元素包含 `image_id` 和 `embedding` 字段），按图片顺序（`image_id` 从小到大）逐一遍历所有人脸。

业务作用：跨图匹配是批量清洗模式的核心能力，确保同一人在不同照片中被分配相同的 `person_id`，使统计报告能够准确反映每个人的出现频次和座位分布。

FR-REC-007 (强需求)：匹配过程中必须维护一个 `person_db` 字典（键为 `person_id` 正整数，值为该人员已知特征向量的列表）。`person_id` 从 1 开始递增。对每个待匹配的人脸，计算其特征向量与 `person_db` 中每个已知身份的最新特征向量的余弦相似度，取最高值。若最高相似度 $\geq$ 相似度阈值（默认 0.5），则分配该 `person_id`；否则创建新的 `person_id` 并加入 `person_db`。

业务作用：贪心最近邻匹配策略在保持实现简洁性的同时，能够在标准车载场景中达到 80% 以上的跨图身份一致率。

FR-REC-008 (强需求)：`person_db` 的生命周期必须限定在单次批量处理请求内。每次调用 `match_faces_across_images` 方法时初始化新的 `person_db`，请求结束后自动清除，不在不同请求之间共享身份信息。

业务作用：避免不同批次的图像数据之间产生身份污染，确保每次批量处理结果的独立性和可重复性。

3.3 座位区域分类规则引擎需求

3.3.1 归一化坐标区域划分规则实现需求

FR-SEAT-001（强需求）：系统必须实现基于归一化坐标的座位区域分类规则引擎。分类依据为人脸边界框中心点的归一化坐标 (nx, ny)，其中 $nx = (x1 + x2) / 2 / \text{图像宽度}$ ， $ny = (y1 + y2) / 2 / \text{图像高度}$ ，取值范围均为 $\backslash[0, 1\backslash]$ 。

业务作用：通过简单的坐标规则引擎实现座位分类，无需训练额外的分类模型，计算开销极低。

FR-SEAT-002（强需求）：系统必须预定义以下五个座位区域，坐标区间必须严格按照下表执行，不得修改：

座位 ID	中文标签	X 范围 (nx)	Y 范围 (ny)
posterior_left	后排左	0.00 \~ 0.33	0.00 \~ 0.50
posterior_right	后排中	0.33 \~ 0.66	0.00 \~ 0.50
after_the_middle	后排右	0.66 \~ 1.00	0.00 \~ 0.50
co_driver	副驾	0.00 \~ 0.50	0.50 \~ 1.00
master_driver	主驾	0.50 \~ 1.00	0.50 \~ 1.00

业务作用：五个座位区域覆盖了标准五座车辆的全部乘员位置。Y 轴上方 (ny 值较小) 对应车辆后排，Y 轴下方 (ny 值较大) 对应车辆前排。

FR-SEAT-003（强需求）：座位区域配置必须作为模块级常量（SEAT_ZONES 字典）定义在 seat_classifier.py 中，可通过 GET /api/seat-zones 接口对外暴露，支持客户端动态查询。

业务作用：将配置集中管理，便于后续扩展为可通过前端界面调整的动态配置。

3.3.2 5 类座位识别、ID 标记、对应颜色绘制需求

FR-SEAT-004（强需求）：系统必须为每个座位区域分配固定的标注颜色，颜色映射关系如下表所示，不得修改：

座位 ID	RGB 颜色值	色相描述
posterior_left	(255, 100, 100)	浅红色
posterior_right	(100, 255, 100)	浅绿色

座位 ID	RGB 颜色值	色相描述
after_the_middle	(100, 100, 255)	浅蓝色
co_driver	(255, 200, 100)	浅橙色
master_driver	(100, 200, 255)	浅青色

业务作用：颜色区分使用户在查看标注图像时能够直观识别各座位区域，无需逐条阅读文字标签。

FR-SEAT-005 (强需求)：系统必须同时维护 RGB 颜色映射（`COLOR_MAP_RGB`）和 BGR 颜色映射（`COLOR_MAP_BGR`，由 RGB 映射自动反转通道生成），分别用于 PIL 标注和 OpenCV 操作。

业务作用：PIL 使用 RGB 色彩空间，OpenCV 使用 BGR 色彩空间，双映射确保在两种图像处理库中标注颜色一致。

FR-SEAT-006 (强需求)：座位分类结果必须包含座位 ID（英文标识字符串，如 "master_driver"）和座位标签（中文名称字符串，如 "主驾"）两个字段。

业务作用：英文 ID 用于程序间数据交换，中文标签用于面向用户的界面展示，双字段设计兼顾了机器可读性和人类可读性。

3.3.3 人脸中心点坐标判定逻辑需求

FR-SEAT-007 (强需求)：座位分类方法 `SeatClassifier.classify` 必须为静态方法，输入参数为人脸边界框 `[x1, y1, x2, y2]`、图像宽度 `img_w` 和图像高度 `img_h`，输出为座位 ID 字符串。判定逻辑为按 `SEAT_ZONES` 字典的遍历顺序，依次检查归一化中心坐标是否落入各区域的 `x_range` 和 `y_range` 范围内，返回第一个匹配区域的 `seat_id`。

业务作用：静态方法设计使分类逻辑无状态、可独立测试。

FR-SEAT-008 (强需求)：当归一化中心坐标未落入任何预定义区域时（理论上不应发生，但为防御性编程），系统必须返回默认值 `"master_driver"`。

业务作用：防止因浮点精度边界问题导致分类返回空值，确保每个人脸都有座位标签。

3.4 后端服务 FastAPI 模块需求

3.4.1 RESTful 标准接口服务需求

FR-API-001 (强需求)：系统必须基于 FastAPI 框架构建 RESTful API 服务，使用 Pydantic 模型定义请求和响应的数据结构。FastAPI 应用实例的元数据必须为：`title= "多人脸采集素材自动化清洗"`，`description= "YOLOv8-face + InsightFace + 规则座位分类"`，`version= "1.0.0"`。

业务作用：FastAPI 原生支持 OpenAPI 文档自动生成和 Pydantic 请求验证，提升开发效率和接口规范性。

FR-API-002 (强需求)：系统必须实现以下五个 API 端点：

方法	路径	功能描述
GET	/	返回前端单页应用 (index.html)
GET	/api/health	返回系统健康状态
POST	/api/detect	单张图片人脸检测
POST	/api/detect-batch	批量图片人脸检测与清洗
GET	/api/seat-zones	返回座位区域配置信息

业务作用：标准化的 API 设计便于前端调用、第三方集成和自动化测试。

FR-API-003 (强需求)：Pydantic 响应模型必须包含以下定义。FaceResult 模型：`bbox: list[float]`、`conf: float`、`seat: str`、`seat_label: str`、`person_id: int`。DetectionResponse 模型：`image_id: int`、`filename: str`、`num_faces: int`、`faces: list[FaceResult]`、`annotated_image_base64: str`、`processing_time_ms: float`。SeatZoneConfig 模型：`zones: dict`。

业务作用：Pydantic 模型自动完成响应数据的序列化和类型校验，确保 API 输出格式的严格一致性。

3.4.2 Uvicorn 服务启动、全网访问、8000 端口需求

FR-API-004 (强需求)：系统必须使用 Uvicorn ASGI 服务器启动 FastAPI 应用。默认启动配置为：`host= "0.0.0.0"`（监听所有网络接口，支持局域网内其他设备访问）、`port= 8000`、`reload= False`（生产模式，不启用热重载）、`log_level= "info"`。

业务作用：0.0.0.0 监听使系统不仅可在本机访问，还可供同一局域网内的其他设备（如手机、平板）通过 IP 地址访问，便于移动端测试。

FR-API-005 (强需求)：backend 包的 `__init__.py` 必须声明版本号 `__version__ = "1.0.0"`，供健康检查接口引用。

业务作用：集中管理版本号，确保系统各接口和文档中的版本信息一致。

3.4.3 静态文件托管、前端页面路由分发需求

FR-API-006 (强需求)：系统必须将 `frontend/` 目录挂载为静态文件服务，挂载路径为 `/static`。GET `/` 路由必须读取 `frontend/index.html` 文件并以 `HTMLResponse` 形式返回。

业务作用：前后端一体化部署，用户无需单独启动前端开发服务器，访问后端地址即可使用完整应用。

FR-API-007 (强需求)：前端文件路径 (`BASE_DIR`) 必须通过 `Path(__file__).resolve().parent.parent.parent` 动态计算，从 `main.py` 的文件位置

(backend/app/main.py) 向上推导三级至 face-detect-app/ 目录根。

业务作用：动态路径计算使系统可在任意安装目录下正确运行，不依赖硬编码路径。

3.4.4 文件上传存储、检测结果持久化存储需求

FR-API-008 (强需求)：系统必须在 data/uploads/ 目录下保存用户上传的原始图片文件，文件名格式为 {当前时间戳毫秒数}{原始文件扩展名}。目录不存在时必须自动创建 (mkdir(parents=True, exist_ok=True))。

业务作用：持久化存储原始图片便于后续复查和数据溯源。

FR-API-009 (强需求)：系统必须在 data/results/ 目录下保存检测生成的标注结果图片。单图检测的结果文件名为 result_{时间戳毫秒数}{扩展名}，批量检测的结果文件名为 batch_{时间戳毫秒数}_{图片序号}.jpg。标注图像的 JPEG 编码质量参数必须为 85 (cv2.IMWRITE_JPEG_QUALITY, 85)。

业务作用：结果持久化存储便于用户离线查看和对比分析。JPEG 质量 85 在文件大小和图像质量之间取得平衡。

FR-API-010 (强需求)：当上传的文件无法被 OpenCV 解码为有效图像时 (cv2.imdecode 返回 None)，系统必须返回 HTTP 400 错误，错误消息为 "无法解析图片文件"。当批量上传中未找到任何有效图片时，系统必须返回 HTTP 400 错误，错误消息为 "未找到有效图片"。

业务作用：明确的错误响应帮助用户快速识别输入问题，避免系统因无效输入而崩溃。

3.4.5 全流程 Pipeline 调度编排接口需求

FR-API-011 (强需求)：系统必须在应用启动时初始化一个全局 Pipeline 实例，初始化参数为 detector_conf=0.5 (检测置信度阈值) 和 similarity_threshold=0.5 (身份匹配相似度阈值)。所有 API 端点共享该 Pipeline 实例。

业务作用：全局单例模式避免每次请求重复初始化模型，显著降低请求处理延迟。

FR-API-012 (强需求)：健康检查接口 GET /api/health 必须返回 {"status": "ok", "message": "多人脸检测服务运行中"}。

业务作用：供监控系统或运维脚本定期探活，确认服务可用性。

FR-API-013 (强需求)：座位区域查询接口 GET /api/seat-zones 必须返回完整的座位区域配置信息，包括每个区域的坐标范围和颜色配置。

业务作用：支持客户端动态获取座位配置，为未来的可视化校准工具提供数据接口。

3.5 前端可视化界面需求

3.5.1 纯原生 HTML5 + CSS3 + Vanilla JS 无框架实现需求

FR-FE-001 (强需求)：前端界面必须以单个 HTML 文件（`frontend/index.html`）形式实现，内嵌 CSS 样式和 JavaScript 脚本，不依赖任何前端框架（React、Vue 等）或构建工具（Webpack、Vite 等）。HTML 文档语言必须为 `zh-CN`，`viewport` 元标签必须包含 `width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no`。

业务作用：单文件、无依赖的前端方案极大简化了部署流程，无需 npm 安装或构建步骤，用户仅需启动后端服务即可使用完整应用。

FR-FE-002 (强需求)：页面标题必须为“多人脸采集素材自动化清洗”。页面头部（header）必须包含标题文字和副标题“YOLOv8-face + InsightFace + 座位分类”。

业务作用：清晰传达系统的功能定位和技术特征。

3.5.2 移动端自适应响应式布局需求

FR-FE-003 (强需求)：前端必须采用移动端优先（Mobile-First）的响应式设计，使用 CSS 媒体查询实现桌面端（宽度 > 480px）和移动端（宽度 ≤ 480px）的自适应布局。页面内容最大宽度限制为 800px，水平居中显示。

业务作用：车载数据采集人员常在车内使用手机或平板操作系统，移动端适配确保系统在这些设备上可正常使用。

FR-FE-004 (强需求)：交互元素（按钮、上传区域）必须满足移动端触控友好性要求，最小可点击区域不小于 44x44 像素。字体必须使用系统字体栈：`-apple-system, "PingFang SC", "Microsoft YaHei", sans-serif`。

业务作用：保证在手机触屏上的操作舒适度和文字可读性。

FR-FE-005 (强需求)：页面整体配色必须为：body 背景色 `#f5f7fa`，header 背景渐变 `linear-gradient(135deg, #1a73e8, #0d47a1)`，上传区域边框色 `#1a73e8`（虚线），上传区域背景色 `#e8f0fe`，圆角 16px。

业务作用：统一的视觉风格提升系统的专业感和品牌辨识度。

3.5.3 单图上传、批量多图上传交互功能需求

FR-FE-006 (强需求)：系统必须提供两个模式切换标签——“单张检测”和“批量清洗”。点击切换时，“单张检测”模式下文件选择器仅允许选择单个文件，“批量清洗”模式下文件选择器允许选择多个文件（HTML input 的 `multiple` 属性动态切换）。文件选择器的 `accept` 属性必须为 `image/*`。

业务作用：模式切换让用户根据实际任务选择合适的工作模式，批量模式的 `multiple` 属性提升多图上传效率。

FR-FE-007 (强需求)：上传区域必须同时支持点击选择文件和拖拽上传文件两种交互方式。拖拽上传必须监听 `dragover`（阻止默认行为并改变样式提示）、`dragleave`（恢复默认样式）、`drop`（阻止默认行为并获取文件列表）三个事件。

业务作用：双模式上传兼顾了桌面端（拖拽更便捷）和移动端（点击更便捷）的用户习惯。

FR-FE-008 (强需求)：处理过程中必须显示全屏加载遮罩层（Loading Overlay），包含旋转动画（CSS `animation: spin 1s linear infinite`）和状态提示文字。遮罩层覆盖整个页面，阻止用户在处理过程中进行重复操作。

业务作用：防止用户因等待时间长而重复点击导致的请求冲突，同时提供视觉反馈表明系统正在工作中。

3.5.4 检测结果图片实时预览、座位色块标注展示需求

FR-FE-009 (强需求)：单张检测结果展示必须包含以下元素：标注图像预览（从 Base64 字符串渲染为 `img` 元素）、人脸数量徽章、处理耗时徽章、人脸详情列表。人脸详情列表中每个人脸必须显示座位标签（带颜色背景标记，颜色通过 JavaScript `getSeatColor` 函数获取）和检测置信度（以百分比进度条形式展示）。

业务作用：结构化的结果展示帮助用户快速理解检测结论，颜色标记和进度条增强了信息的直观性。

FR-FE-010 (强需求)：前端 JavaScript 必须内置以下座位颜色映射和标签映射，与后端保持一致。

颜色映射（`getSeatColor` 函数）：`master_driver` = `#4fc3f7`、`co_driver` = `#ffb74d`、`posterior_left` = `#e57373`、`posterior_right` = `#81c784`、`after_the_middle` = `#64b5f6`，默认值 = `#bdbdbd`。

标签映射（`getSeatLabel` 函数）：`master_driver` = "主驾"、`co_driver` = "副驾"、`posterior_left` = "后排左"、`posterior_right` = "后排中"、`after_the_middle` = "后排右"。

业务作用：前端维护独立的颜色/标签映射，即使后端配置查询接口不可用，前端仍可正确渲染结果。

3.5.5 批量清洗统计报告前端展示需求

FR-FE-011 (强需求)：批量清洗结果展示必须包含以下元素：聚合统计表格（表头为"人员 ID"、"出现次数"、"主要座位"，逐行展示每个 `person_id` 的统计数据）、全局人脸列表（汇总所有图片的人脸信息）、逐图结果卡片（每张图片独立的标注图和人脸列表，结构与单张检测一致）。

业务作用：聚合统计表格是批量模式的核心输出，帮助用户快速掌握"有多少不同的人、每人出现了几次、主要坐在哪里"。

FR-FE-012 (强需求)：页面必须包含座位区域图例卡片，以彩色圆点和中文标签展示五个座位区域的名称和对应颜色。图例颜色配置为：主驾 = `#64b5f6`、副驾 = `#ffb74d`、后排左 = `#e57373`、后排中 = `#81c784`、后排右 = `#64b5f6`。

业务作用：图例卡片帮助用户在查看标注图像时快速建立颜色与座位的对应关系。

3.6 文件与目录管理需求

3.6.1 data/uploads 上传图片存储目录管理

FR-DIR-001 (强需求)：系统必须在 `face-detect-app/data/uploads/` 目录下存储用户上传的原始图片文件。目录不存在时，系统启动阶段必须自动创建（`mkdir(parents=True, exist_ok=True)`）。

业务作用：持久化存储上传文件，支持数据溯源和离线复查。

FR-DIR-002 (强需求)：上传文件的命名必须使用时间戳毫秒数加原始扩展名的格式（例如 `1720345678901.jpg`），避免文件名冲突。

业务作用：时间戳命名确保并发上传时不会产生文件名覆盖。

3.6.2 data/results 检测输出结果存储目录管理

FR-DIR-003 (强需求)：系统必须在 `face-detect-app/data/results/` 目录下存储检测生成的标注结果图片。目录不存在时自动创建。

业务作用：结果图片的持久化存储便于用户离线查看、对比分析和二次使用。

FR-DIR-004 (强需求)：`data/` 目录必须被 `.gitignore` 排除，不纳入版本控制。

业务作用：避免运行时产生的数据文件被意外提交到代码仓库。

3.6.3 models 模型权重文件目录加载管理

FR-DIR-005 (弱需求/可选扩展)：项目目录结构中保留 `models/` 目录，用于存放 YOLOv8-face 模型权重文件（`yolov8n-face.pt`）和 InsightFace ArcFace 模型文件。该目录为可选目录，不存在时系统自动降级至内置方案。

业务作用：为需要使用高精度模型的用户提供标准的模型文件存放位置。

4 非功能性需求

4.1 性能需求

NFR-PERF-001 (强需求)：在 CPU 推理环境下（Intel i5 及以上处理器，8GB 内存），单张标准分辨率图像（1920x1080 像素，包含 5 名乘员）的人脸检测+座位分类+标注处理耗时必须不超过 5000 毫秒。系统基准测试数据为：5 人合成测试图，CPU 单图处理耗时约 300 毫秒。

NFR-PERF-002 (强需求)：批量处理 10 张图像（每张包含 3-5 名乘员）的总处理耗时必须不超过 60 秒。

NFR-PERF-003 (强需求)：前端页面（`index.html`，约 500 行代码）在本地网络环境下的首次加载时间必须不超过 2 秒。

NFR-PERF-004 (强需求)：在基准测试条件下（5 人合成测试图），人脸检测准确率必须达到 100%（5/5 全部检出），座位分类准确率必须达到 100%（5/5 全部分类正确）。

4.2 兼容性需求

NFR-COMP-001 (强需求)：系统必须支持模型降级兼容机制。当 YOLOv8-face 模型不可用时自动切换至 Haar Cascade 引擎，当 InsightFace 模型不可用时自动切换至像素统计特征引擎。降级过程必须自动完成，无需人工干预，不导致服务中断。

NFR-COMP-002 (强需求)：前端界面必须兼容以下现代浏览器：Chrome 90+、Firefox 88+、Edge 90+、Safari 14+。

NFR-COMP-003 (强需求)：后端服务必须兼容以下操作系统：Windows 10/11、macOS 12+、Ubuntu 20.04+。

NFR-COMP-004 (强需求)：系统必须支持 JPEG（.jpg/.jpeg）和 PNG（.png）两种图像格式的输入。

4.3 易用性需求

NFR-USE-001 (强需求)：前端界面无需独立部署或构建。用户仅需启动后端服务，通过浏览器访问 `http://localhost:8000` 即可使用完整功能。

NFR-USE-002 (强需求)：系统必须提供一键式服务启动命令：`python -m uvicorn app.main:app --host 0.0.0.0 --port 8000`，在 backend 目录下执行。

NFR-USE-003 (强需求)：上传区域必须支持拖拽上传和点击上传两种交互方式，处理过程中必须显示加载遮罩防止重复操作。

4.4 可维护性需求

NFR-MAIN-001 (强需求)：后端代码必须按功能模块划分为独立的 Python 文件：`detector.py`（人脸检测引擎）、`recognizer.py`（特征匹配引擎）、`seat_classifier.py`（座位分类与标注引擎）、`pipeline.py`（流水线编排）、`main.py`（API 路由与服务启动）。各文件职责单一，边界清晰。

NFR-MAIN-002 (强需求)：系统配置参数（检测置信度阈值 0.5、身份匹配相似度阈值 0.5、Haar Cascade 检测参数等）必须在代码中集中定义，不得分散在业务逻辑中。

NFR-MAIN-003 (强需求)：系统必须提供标准的 Python 依赖声明文件（`requirements.txt`），包含全部 9 项依赖及其最低版本要求，支持 `pip install -r requirements.txt` 一键安装。

4.5 部署运维需求

NFR-DEP-001 (强需求)：系统必须为轻量化 Python 服务，不依赖 Docker 容器、Kubernetes 编排或其他重型基础设施。直接在 Python 环境中运行即可。

- NFR-DEP-002 (强需求)：**系统不依赖外部数据库服务。运行时数据（person_db、检测结果）全部在内存中管理，请求结束后自动清除。
- NFR-DEP-003 (强需求)：**系统不依赖外部网络服务。全部计算在本地完成，可完全离线运行（在使用降级引擎的情况下）。
- NFR-DEP-004 (强需求)：**系统启动日志必须清晰记录引擎选择信息（如 "YOLOv8-face 模型未找到，将使用 OpenCV Haar Cascade 作为 fallback"、"Haar Cascade 人脸检测器就绪"），便于运维人员确认系统运行状态。

5 数据与规则约束需求

5.1 座位分类坐标硬性规则

座位分类的坐标规则是本系统最核心的业务约束之一，以下为完整的座位区域定义表，系统实现必须严格遵循，不得修改任何坐标区间值。

座位 ID	中文标签	X 范围 (nx 归一化横坐标)	Y 范围 (ny 归一化纵坐标)	RGB 标注颜色	色相描述
posterior_left	后排左	$0.00 \leq nx \leq 0.33$	$0.00 \leq ny \leq 0.50$	(255, 100, 100)	浅红色
posterior_right	后排中	$0.33 < nx \leq 0.66$	$0.00 \leq ny \leq 0.50$	(100, 255, 100)	浅绿色
after_the_middle	后排右	$0.66 < nx \leq 1.00$	$0.00 \leq ny \leq 0.50$	(100, 100, 255)	浅蓝色
co_driver	副驾	$0.00 \leq nx \leq 0.50$	$0.50 < ny \leq 1.00$	(255, 200, 100)	浅橙色
master_driver	主驾	$0.50 < nx \leq 1.00$	$0.50 < ny \leq 1.00$	(100, 200, 255)	浅青色

坐标计算规则：

归一化横坐标 $nx = (bbox_x1 + bbox_x2) / 2 / image_width$

归一化纵坐标 $ny = (bbox_y1 + bbox_y2) / 2 / image_height$

坐标系约定：图像 Y 轴上方（ny 值较小）对应车辆后排，Y 轴下方（ny 值较大）对应车辆前排。X 轴左侧（nx 值较小）对应车辆左侧，X 轴右侧（nx 值较大）对应车辆右侧。

默认降级规则：当归一化中心坐标未落入上述任何一个区域时（理论上由于五个区域已覆盖 $\backslash[0,1\backslash]\times\backslash[0,1\backslash]$ 的全部空间，此情况不应发生），系统返回默认值 "master_driver"。

5.2 人脸检测降级触发规则

人脸检测引擎的降级判定逻辑如下，必须在系统启动阶段（FaceDetector 初始化时）完成，运行时不可动态切换。

优先级	判定条件	执行动作	日志输出
1	<code>from ultralytics import YOLO</code> 成功 且 <code>yolov8n-face.pt</code> 文件在当前目录或 <code>~/.ultralytics/weights/</code> 目录中存在	加载 YOLOv8-face 模型，设置 <code>self.backend = "yolo"</code>	无特殊日志
2	<code>from ultralytics import YOLO</code> 触发 <code>ImportError</code>	进入 Haar Cascade 初始化	<code>"[FaceDetector]</code> <code>ultralytics</code> 未安装， 使用 <code>OpenCV Haar Cascade"</code>
3	<code>yolov8n-face.pt</code> 文件在所有预设路径中均不存在	进入 Haar Cascade 初始化	<code>"[FaceDetector]</code> <code>YOLOv8-face</code> 模型未找到 (<code>{model_file}</code>)，将 使用 <code>OpenCV Haar Cascade</code> 作为 <code>fallback"</code>
4	Haar Cascade 模型文件 <code>haarcascade_frontalface_default.xml</code> 加载失败（ <code>cv2.CascadeClassifier.load()</code> 返回空）	抛出 <code>RuntimeError</code>	异常消息 "无法加载 Haar Cascade 模型： <code>{cascade_path}"</code>
5	Haar Cascade 模型加载成功	设置 <code>self.backend = "haar"</code>	<code>"[FaceDetector]</code> Haar <code>Cascade</code> 人脸检测器就绪"

5.3 特征匹配降级触发规则

人脸特征匹配引擎的降级规则如下。当前版本的代码实现中，recognizer.py 模块直接实现了像素统计特征方案，未实际集成 InsightFace ArcFace 引擎（尽管 requirements.txt 中声明了 `insightface>=0.7.3`

依赖）。

方案	特征维度	特征提取方法	匹配度量	触发条件
主方案 (设计层)	512 维	InsightFace ArcFace 深度学习嵌入	余弦相似度	InsightFace 库已安装且模型已下载
降级方案 (当前实现)	256 维	人脸 ROI → 16x16 缩放 → 灰度化 → 直方图均衡化 → 展平 → L2 归一化	余弦相似度 (归一化向量的点积)	默认方案，无需额外依赖

特征匹配相似度阈值默认值为 0.5。当两张人脸的余弦相似度 ≥ 0.5 时判定为同一人，否则判定为不同人。该阈值在 pipeline.py 中以硬编码形式传入 `match_faces_across_images` 方法。

5.4 图片输入输出格式约束

输入约束：

约束项	要求
文件格式	仅接受 JPEG (.jpg/.jpeg) 和 PNG (.png) 格式
文件格式校验方式	通过 OpenCV <code>cv2.imdecode</code> 解码结果判断，返回 None 则为无效格式
色彩空间	解码后为 BGR 三通道 (OpenCV 默认)
分辨率限制	无硬性限制，建议不超过 3840x2160
文件大小限制	建议不超过 20MB

输出约束：

约束项	要求
标注图像格式	JPEG 编码，质量参数 85 (<code>cv2.IMWRITE_JPEG_QUALITY, 85</code>)
标注图像传输方式	Base64 编码嵌入 JSON 响应，包含 data URI 前缀 <code>data:image/jpeg;base64,</code>
结构化数据格式	JSON，字段定义遵循 Pydantic 模型 <code>FaceResult</code> 和 <code>DetectionResponse</code>
持久化存储格式	标注图片以 JPEG 格式保存至 <code>data/results/</code> 目录

6 系统模块与对应代码文件映射说明

根据项目的完整目录结构，以下逐一说明 `backend/app/` 下各核心文件及 `frontend/` 下前端文件所承载的需求功能。

6.1 backend/app/detector.py — 人脸检测引擎

承载需求：FR-DET-001 至 FR-DET-010（人脸检测模块全部需求）。

核心类：`FaceDetector`。

职责说明：该文件实现了人脸检测的全部逻辑。类初始化时（`__init__`）接受可选参数 `model_path`（模型文件路径）和 `conf_threshold`（置信度阈值，默认 0.5），调用 `_init_detector` 方法执行引擎选择。`_init_detector` 按优先级尝试加载 YOLOv8-face 模型，失败时降级至 Haar Cascade。

`_init_haar` 方法加载 OpenCV 内置的 `haarcascade_frontalface_default.xml` 分类器。

`detect` 方法是对外暴露的唯一检测接口，内部根据 `self.backend` 属性分派至 `_detect_yolo` 或 `_detect_haar`。YOLO 检测方法调用 `self.yolo_model.predict(source=image, conf=self.conf_threshold, verbose=False, device="cpu")`，提取边界框和置信度。Haar 检测方法先转灰度图，再调用 `detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(60, 60), flags=cv2.CASCADE_SCALE_IMAGE)`，伪置信度通过 `min(0.5 + area_ratio * 10, 0.99)` 计算。两种引擎的输出格式统一为 `[{"bbox": [x1, y1, x2, y2], "conf": float}, ...]`，按 `bbox` 的 `x1` 坐标从左至右排序。

`is_yolo` 属性返回布尔值，标识当前使用的检测引擎类型。

6.2 backend/app/recognizer.py — 人脸特征匹配引擎

承载需求：FR-REC-001 至 FR-REC-008（人脸特征匹配模块全部需求）。

核心类：`FaceRecognizer`。

职责说明：该文件实现了人脸特征提取和跨图片身份匹配的全部逻辑。类初始化时设置 `self.available = True` 和延迟加载的 Haar Cascade 引用 `_lbph_cascade = None`。

`extract_embedding` 方法接受图像数组和边界框坐标，裁剪人脸 ROI 区域（带边界钳位），缩放至 16x16，转灰度，执行 `cv2.equalizeHist` 直方图均衡化，展平为 `float32` 数组（256 维），L2 归一化。归一化时若向量范数 $\leq 1e-6$ 则跳过，避免除零错误。ROI 为空时返回 `None`。

`cosine_similarity` 为静态方法，计算两个归一化向量的点积作为余弦相似度，任一输入为 `None` 时返回 0.0。

`match_faces_across_images` 方法接受包含全部图片人脸信息的列表和相似度阈值（默认 0.5），按 `image_id` 排序后逐一遍历，维护 `person_db` 字典实现贪心最近邻匹配。`person_id` 从 1 开始递增，匹配结果直接修改输入列表中各元素的 `person_id` 字段。

6.3 backend/app/seat_classifier.py — 座位分类与可视化标注引擎

承载需求：FR-SEAT-001 至 FR-SEAT-008（座位分类模块全部需求）以及可视化标注功能。

核心类：SeatClassifier（座位分类器）、FaceAnnotator（标注绘制器）。

模块级常量：

SEAT_ZONES 字典定义了五个座位区域的坐标范围和中文标签，内容与第 5.1 节完全一致。

COLOR_MAP_RGB 字典定义了五个座位区域的 RGB 标注颜色。COLOR_MAP_BGR 字典通过 `{k: (v[2], v[1], v[0]) for k, v in COLOR_MAP_RGB.items()}` 自动生成，用于 OpenCV 操作。

FONT_PATHS 列表定义了中文渲染字体的查找路径：`C:/Windows/Fonts/simhei.ttf`、`C:/Windows/Fonts/msyh.ttc`、`C:/Windows/Fonts/simsun.ttc`。_get_font 函数按顺序尝试加载，全部失败时回退至 `ImageFont.load_default()`。

SeatClassifier 类的 classify 为静态方法，计算归一化中心坐标后遍历 SEAT_ZONES 进行区域匹配，默认返回 "master_driver"。get_label 返回座位 ID 对应的中文标签。get_color 返回座位 ID 对应的 BGR 颜色，默认值为 (200, 200, 200)。

FaceAnnotator 类的 draw 为静态方法，将 OpenCV BGR 图像转换为 PIL RGB 图像，使用 ImageDraw 绘制矩形框（线宽 3 像素，颜色来自 COLOR_MAP_RGB）和中文标签（字体大小 18，标签格式为 "{座位标签} ID:{person_id}"，白色文字绘制在填充色背景条上），置信度文字（格式 "{conf:.2f}"）绘制在边界框下方。完成后转回 OpenCV 格式返回。draw_seat_zones 为静态方法，在图像上绘制半透明的座位区域覆盖层（alpha=0.15），用于可视化展示区域划分。

6.4 backend/app/pipeline.py — 流水线编排引擎

承载需求：FR-API-011（Pipeline 全局实例）、批量处理四阶段流程。

核心类：Pipeline。

职责说明：该文件是系统的流程编排中枢，整合 detector、recognizer、seat_classifier 三个核心模块。类初始化时创建 FaceDetector（conf_threshold=detector_conf，默认 0.5）、FaceRecognizer、SeatClassifier 和 FaceAnnotator 四个引擎的单例引用。

process_image 方法处理单张图像：检测 → 逐脸提取特征 → 身份匹配（硬编码 similarity_threshold=0.5） → 逐脸座位分类 → 标注绘制。返回字典包含 image_id、num_faces、faces 列表（每个元素包含 bbox、conf、seat、seat_label、person_id）和 annotated_image。

process_batch 方法处理批量图像：阶段一逐图检测并收集全部人脸（标记 image_id）；阶段二调用 match_faces_across_images 进行全局身份匹配（硬编码 similarity_threshold=0.5）；阶段三按 image_id 拆分结果并逐图执行座位分类和标注；阶段四返回 (results_list, annotated_images) 元组。

6.5 backend/app/main.py — API 路由与服务启动

承载需求：FR-API-001 至 FR-API-013（后端服务模块全部需求）、FR-DIR-001 至 FR-DIR-004（文件目录管理需求）。

职责说明：该文件定义了 FastAPI 应用实例、五个 API 端点和文件存储逻辑。模块级代码计算 BASE_DIR（通过 `Path(__file__).resolve().parent.parent.parent` 推导至 `face-detect-app/` 目录），创建 UPLOAD_DIR（`data/uploads/`）和 RESULT_DIR（`data/results/`），初始化全局 Pipeline 实例（`detector_conf=0.5, similarity_threshold=0.5`），配置静态文件挂载（`/static` → `frontend/` 目录）。

Pydantic 模型 `FaceResult`、`DetectionResponse`、`SeatZoneConfig` 定义了 API 响应的数据结构。五个路由方法分别实现首页服务、健康检查、单图检测、批量检测和座位配置查询。单图检测和批量检测接口内包含文件保存逻辑（原始文件和标注结果分别保存至 `uploads` 和 `results` 目录）。`__main__` 块中以 `uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=False, log_level="info")` 启动服务。

6.6 frontend/index.html — 前端单页应用

承载需求：FR-FE-001 至 FR-FE-012（前端界面模块全部需求）。

职责说明：该文件是系统唯一的前端文件，约 500 行代码，内嵌 CSS 和 JavaScript。HTML 结构包含标题栏、座位图例卡片、模式切换标签、文件上传区域、加载遮罩层和结果展示区域。CSS 实现了移动端优先的响应式布局（480px 断点）、渐变头部、虚线上传区域等视觉样式，以及五个座位区域的独立背景色 CSS 类。JavaScript 实现了模式切换（`switchMode`）、文件处理分发（`handleFiles`）、单图检测请求（`handleSingle`，`POST /api/detect`）、批量检测请求（`handleBatch`，`POST /api/detect-batch`）、单图结果渲染（`renderSingleResult`）、批量结果渲染（`renderBatchResult`）、拖拽上传事件监听、颜色和标签映射（`getSeatColor`、`getSeatLabel`）等全部交互逻辑。

7 系统部署与使用流程需求

7.1 环境依赖安装需求

FR-DEP-001（强需求）：系统运行需要 Python 3.9 或更高版本。以下 9 项 Python 依赖必须通过 `pip install -r requirements.txt` 安装，各依赖的最低版本要求如下表所示：

序号	依赖包名	最低版本	功能说明
1	fastapi	\>= 0.104.0	Web 服务框架
2	uvicorn\[standard\]	\>= 0.24.0	ASGI 服务器

序号	依赖包名	最低版本	功能说明
3	ultralytics	\>= 8.0.0	YOLOv8-face 人脸检测（主方案）
4	insightface	\>= 0.7.3	ArcFace 人脸特征匹配（主方案，可选）
5	opencv-python	\>= 4.8.0	图像处理、Haar Cascade（降级方案）
6	numpy	\>= 1.24.0	数值计算
7	pydantic	\>= 2.0.0	请求/响应数据校验
8	python-multipart	\>= 0.0.6	multipart/form-data 文件上传解析
9	Pillow	\>= 10.0.0	PIL 图像标注绘制、中文字体渲染

FR-DEP-002（弱需求/可选扩展）：ultralytics 和 insightface 为可选依赖。若安装失败或版本不满足，系统自动降级至内置方案（Haar Cascade 检测引擎和像素统计特征匹配引擎），不影响系统基本运行。

7.2 服务启动操作流程需求

FR-DEP-003（强需求）：服务启动必须在 `face-detect-app/backend/` 目录下执行以下命令：

```
cd face-detect-app/backend
pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

启动成功后，终端将显示 Uvicorn 的启动日志，包含 `"Uvicorn running on http://0.0.0.0:8000"` 信息。此时服务已就绪，可接受客户端请求。

FR-DEP-004（强需求）：如需停止服务，在终端中按 `Ctrl+C`。如需更改端口号，修改 `--port` 参数值即可（如 `--port 8001`）。如需更改模型置信度阈值，需修改 `main.py` 中 Pipeline 初始化参数。

7.3 前端页面访问使用流程需求

FR-DEP-005（强需求）：服务启动后，用户在任意现代浏览器中访问 `http://localhost:8000` 即可打开系统前端界面。若服务启动时使用了自定义端口，则相应修改 URL 中的端口号。局域网内其他设备可通过运行服务的主机 IP 地址访问（如 `http://192.168.1.100:8000`）。

FR-DEP-006（强需求）：用户操作流程为：打开浏览器 → 访问系统地址 → 选择工作模式（单张检测或批量清洗） → 上传一张或多张图片 → 等待处理完成 → 查看标注结果和统计报告。无需安装任何客户端软件或浏览器插件。

8 系统验收标准

8.1 人脸检测验收指标

验收项	测试条件	验收标准	优先级
检出率	5 人合成测试图（标准光照、正面角度）	5/5 全部检出，检出率 100%	强需求
多人脸支持	包含 5 名及以上乘员的图像	系统正确返回 ≥ 5 个人脸检测结果	强需求
坐标有效性	任意测试图像	每个 bbox 满足 $0 \leq x1 < x2 \leq \text{图像宽度}$, $0 \leq y1 < y2 \leq \text{图像高度}$	强需求
置信度有效范围	任意测试图像	每个 conf 值满足 $0 < \text{conf} \leq 1$	强需求
排序正确性	多人图像	faces 数组按 bbox 的 x1 坐标从左至右排列	强需求
降级检测	移除 YOLOv8-face 模型文件	Haar Cascade 引擎正常启动并返回有效检测结果	强需求

8.2 座位分类验收指标

验收项	测试条件	验收标准	优先级
分类准确率	5 人合成测试图（标准摄像头视角）	5/5 全部分类正确，准确率 100%	强需求
五区域覆盖	分别覆盖五个座位区域的测试图像集	每个区域均有正确分类结果	强需求
颜色一致性	任意测试图像	标注框颜色与 5.1 节定义的颜色映射完全一致	强需求
中文标签	任意测试图像	标注图像上的座位名称中文文字正确渲染（无乱码）	强需求

验收项	测试条件	验收标准	优先级
配置查询	调用 GET /api/seat-zones	返回的坐标区间与 5.1 节定义完全一致	强需求

8.3 处理性能验收指标

验收项	测试条件	验收标准	优先级
单图处理耗时	5 人合成测试图，CPU 环境（Intel i5，8GB RAM）	处理耗时约 300ms，不超过 5000ms	强需求
批量处理耗时	10 张图像（每张 3-5 人），CPU 环境	总耗时不超过 60 秒	强需求
前端加载时间	本地网络环境	首屏加载不超过 2 秒	强需求
API 响应时间	单张 1920x1080 图像，含网络传输	端到端响应不超过 8 秒	强需求

8.4 功能验收清单

序号	功能项	验收方法	通过标准	优先级
1	单图检测	上传一张多人图片至 POST /api/detect	返回有效 JSON 响应，num_faces > 0，annotated_image_base64 可解码为有效图像	强需求
2	批量清洗	上传 3 张图片至 POST /api/detect-batch	返回 total_images=3，results 数组长度=3，aggregate_stats 非空	强需求
3	跨图身份匹配	上传 2 张包含同一人的图片	该人在两张图中 person_id 一致	强需求

序号	功能项	验收方法	通过标准	优先级
4	座位标注显示	查看标注图像	每个座位区域使用不同颜色框标注，中文标签清晰可见	强需求
5	模式切换	在前端点击"单张检测"和"批量清洗"	界面正确切换，文件选择器 multiple 属性相应变化	强需求
6	拖拽上传	在桌面端浏览器中拖拽图片到上传区域	文件被接收并开始处理	强需求
7	移动端适配	在 375px 宽度视口中访问	页面布局正常，交互元素可操作	强需求
8	健康检查	调用 GET /api/health	返回 {"status": "ok", "message": "多人脸检测服务运行中"}	强需求
9	错误处理	上传 .txt 文件至 POST /api/detect	返回 HTTP 400 和错误消息，服务不崩溃	强需求
10	文件持久化	上传并检测一张图片	data/uploads/ 和 data/results/ 目录中出现对应文件	强需求
11	引擎降级	移除所有可选模型后启动服务	系统正常启动，日志显示降级信息，功能可用	强需求
12	前端结果展示	单图检测后查看结果页	标注图、人脸数量、处理耗时、人脸列表均正确显示	强需求

9 附录

9.1 整体技术架构对照表

技术层次	主方案实现	降级方案实现	核心技术/库	部署位置
人脸检测	YOLOv8-face	OpenCV Haar Cascade	ultralytics / cv2.CascadeClassifier	backend/app/detector.py
人脸特征匹配	InsightFace ArcFace 512 维	像素统计 特征 256 维	insightface / cv2 + numpy	backend/app/recognizer.py
座位区域分类	坐标规则引擎（5 区域）	无降级 （规则引擎为完整 方案）	numpy 坐标计算	backend/app/seat_classifier.py
可视化标注	PIL 图像绘制 + 中文字体	无降级	PIL.ImageDraw + ImageFont	backend/app/seat_classifier.py
流水线编排	同步顺序执行	无降级	Python 类组合	backend/app/pipeline.py

技术层次	主方案实现	降级方案实现	核心技术/库	部署位置
后端服务	FastAPI + Uvicorn	无降级	fastapi + uvicorn	backend/app/main.py
前端界面	HTML5 + CSS3 + Vanilla JS	无降级	原生浏览器 API	frontend/index.html
数据校验	Pydantic 模型	无降级	pydantic v2	backend/app/main.py
文件上传	multipart/form-data	无降级	python-multipart	backend/app/main.py

9.2 项目完整目录树形结构

```
face-detect-app/
├── backend/
│   ├── __init__.py           # 包初始化，声明版本号 __version__ = "1.0.0"
│   ├── requirements.txt      # Python 依赖声明（9 项依赖）
│   └── app/
│       ├── __init__.py      # 应用包初始化（空文件）
│       └── main.py          # FastAPI 应用入口、API 路由、静态文件服务、Uvicorn
启动
│   └── detector.py          # 人脸检测引擎（YOLOv8-face 主方案 + Haar Cascade 降
级）
│   └── recognizer.py        # 人脸特征匹配引擎（像素统计特征 + 跨图身份匹配）
│   └── seat_classifier.py    # 座位区域分类器 + 可视化标注器（规则引擎 + PIL 绘
制）
```

└─ pipeline.py	# 流水线编排引擎（单图处理 + 批量处理四阶段流程）
└─ frontend/	
└─ index.html	# 前端单页应用（HTML5 + CSS3 + JS，约 500 行）
└─ docs/	
└─ 需求规格说明书.md	# 需求规格说明文档
└─ 概要设计文档.md	# 概要设计文档
└─ data/	# 运行时数据目录（.gitignore 排除）
└─ uploads/	# 用户上传原始图片存储
└─ results/	# 检测标注结果图片存储
└─ models/	# 模型权重文件目录（可选）
└─ (yolov8n-face.pt)	# YOLOv8-face 模型文件（可选，不存在时自动降级）

仓库级目录结构补充说明：

face-detect/	# 仓库根目录
└─ .gitignore	# Git 忽略规则（Python 缓存、venv、IDE 文件、data/ 目录等）
└─ S0512333 软件开发实践3-考核作业要求即任务书-2026.docx	# 课程任务书
└─ 第五组_任务分工分配表.docx	# 团队分工文档
└─ face-detect-app/	# 核心应用目录（即上述目录结构）

9.3 项目技术栈清单

类别	技术/工具	版本要求	用途
编程语言	Python	\>= 3.9	后端开发语言
Web 框架	FastAPI	\>= 0.104.0	RESTful API 服务框架
ASGI 服务器	Uvicorn	\>= 0.24.0	高性能异步 Web 服务器
人脸检测 (主)	ultralytics (YOLOv8-face)	\>= 8.0.0	深度学习人脸检测引擎
人脸检测 (降级)	OpenCV Haar Cascade	\>= 4.8.0	传统人脸检测分类器
人脸匹配 (主)	InsightFace (ArcFace)	\>= 0.7.3	深度学习人脸特征提取

类别	技术/工具	版本要求	用途
人脸匹配 (降级)	OpenCV + NumPy	\>= 4.8.0 / >= 1.24.0	像素统计特征提取
图像处理	OpenCV (opencv-python)	\>= 4.8.0	图像解码、灰度转换、直方图均衡化
数值计算	NumPy	\>= 1.24.0	向量运算、矩阵操作
数据校验	Pydantic	\>= 2.0.0	API 请求/响应模型定义与校验
文件上传	python-multipart	\>= 0.0.6	multipart/form-data 解析
图像标注	Pillow	\>= 10.0.0	PIL 图像绘制、中文字体渲染
前端技术	HTML5 + CSS3 + JavaScript	无版本要求 (现代浏览器)	响应式单页应用界面
版本控制	Git	无版本要求	代码版本管理

--